

Tentu, mari kita bedah kode `PseudoColorHelper` ini. Kode ini adalah bagian dari logika pemrosesan gambar (image processing) tingkat rendah (low-level) yang memanipulasi *byte* pixel secara langsung untuk kecepatan maksimal.

Secara garis besar, kelas ini memiliki dua fungsi utama:

1. **Pewarnaan Semu (Pseudo Color):** Mengubah gambar menjadi grayscale, lalu memetakannya ke palet warna (seperti efek *Thermal* atau *Sepia*).
2. **Isolasi Objek (Magic Wand):** Menyeleksi warna tertentu, membuat garis batas putus-putus (*marching ants*), memotong objek, dan menghapus objek dari background.

Berikut adalah penjelasan detail per bagian dan per baris:

Bagian 1: Persiapan & Enum

C#

```
using System;
using System.Drawing;
using System.Drawing.Imaging; // Penting untuk LockBits & BitmapData
using System.Runtime.InteropServices; // Penting untuk Marshal.Copy
using System.Threading.Tasks; // Penting untuk Parallel.For (Multi-core processing)

namespace MiniPhotoshop.Logic.Helpers
{
    public class PseudoColorHelper
    {
        public enum ColorMapType
        {
            Hot, // Hitam -> Merah -> Kuning -> Putih (Thermal)
            Cool, // Cyan -> Magenta
            Jet, // Biru -> Hijau -> Merah (Spektrum Pelangi)
            Gold // Efek Emas/Sepia
        }
    }
}
```

- **Imports:** Kita menggunakan `System.Drawing.Imaging` untuk akses data bit gambar dan `System.Threading.Tasks` untuk mempercepat proses menggunakan semua inti CPU (multithreading).
- **Enum `ColorMapType`:** Daftar pilihan efek warna yang tersedia bagi pengguna.

Bagian 2: Fungsi ApplyPseudoColor

Fungsi ini mengubah gambar menjadi grayscale, lalu setiap nilai abu-abu diganti dengan warna dari tabel (palette).

C#

```
public Bitmap ApplyPseudoColor(Bitmap source, ColorMapType type)
{
    if (source == null) return null; // Cek null safety

    // Mendapatkan dimensi gambar
    int w = source.Width;
    int h = source.Height;
    // Membuat bitmap kosong untuk hasil (dst = destination)
    Bitmap dst = new Bitmap(w, h, PixelFormat.Format32bppArgb);

    // --- TEKNIK LOCKBITS (Sangat Penting) ---
    // Mengunci memori gambar agar kita bisa membaca/menulis byte secara langsung.
    // Jauh lebih cepat daripada menggunakan GetPixel/SetPixel.
    BitmapData srcData = source.LockBits(new Rectangle(0, 0, w, h),
    ImageLockMode.ReadOnly, PixelFormat.Format32bppArgb);
    BitmapData dstData = dst.LockBits(new Rectangle(0, 0, w, h), ImageLockMode.WriteOnly,
    PixelFormat.Format32bppArgb);

    // Menghitung total byte. Stride adalah lebar baris memori (kadang ada padding).
    int bytes = Math.Abs(srcData.Stride) * h;
    byte[] srcBuffer = new byte[bytes]; // Wadah data sumber
    byte[] dstBuffer = new byte[bytes]; // Wadah data hasil

    // Menyalin data dari memori unmanaged (Scan0) ke array byte managed (srcBuffer)
    Marshal.Copy(srcData.Scan0, srcBuffer, 0, bytes);

    // Membuat tabel warna (Palette) berdasarkan tipe yang dipilih
    byte[][] palette = CreatePalette(type);

    // --- PARALLEL PROCESSING ---
    // Melakukan loop menggunakan semua core CPU.
    // bytes / 4 karena 1 pixel = 4 byte (B, G, R, Alpha).
```

```

Parallel.For(0, bytes / 4, i =>
{
    int idx = i * 4; // Index awal pixel saat ini

    // Membaca warna asli (Format urutan byte biasanya B-G-R-A)
    byte b = srcBuffer[idx];
    byte g = srcBuffer[idx + 1];
    byte r = srcBuffer[idx + 2];

    // Mengubah ke Grayscale rata-rata sederhana
    byte gray = (byte)((r + g + b) / 3);

    // Mengambil warna baru dari palette berdasarkan tingkat keabuan (0-255)
    dstBuffer[idx] = palette[gray][0]; // Blue baru
    dstBuffer[idx + 1] = palette[gray][1]; // Green baru
    dstBuffer[idx + 2] = palette[gray][2]; // Red baru
    dstBuffer[idx + 3] = 255; // Alpha (Tidak transparan)
});

// Menyalin kembali data hasil ke Bitmap tujuan
Marshal.Copy(dstBuffer, 0, dstData.Scan0, bytes);

// Melepas kunci memori (Wajib dilakukan agar memori tidak bocor/error)
source.UnlockBits(srcData);
dst.UnlockBits(dstData);

return dst;
}

```

Bagian 3: Fungsi CreatePalette

Fungsi ini menghitung matematika warna untuk membuat gradasi.

C#

```

private byte[][] CreatePalette(ColorMapType type)
{
    // Membuat array 256 baris (0-255), masing-masing baris punya 3 slot (B, G, R)

```

```

byte[][] palette = new byte[256][];
for (int i = 0; i < 256; i++)
{
    palette[i] = new byte[3];
    double val = i / 255.0; // Normalisasi nilai i menjadi 0.0 sampai 1.0

    switch (type)
    {
        case ColorMapType.Hot:
            // Logika matematika untuk gradasi Hitam->Merah->Kuning->Putih
            palette[i][2] = (byte)(Math.Min(1, val * 3) * 255); // Red naik duluan
            palette[i][1] = (byte)(Math.Min(1, Math.Max(0, (val - 0.33) * 3)) * 255); // Green menyusul
            palette[i][0] = (byte)(Math.Min(1, Math.Max(0, (val - 0.66) * 3)) * 255); // Blue terakhir
            // (untuk jadi putih)
            break;
        // ... (Logic case Jet dan Gold mirip, hanya beda rumus matematika warnanya) ...
    }
}
return palette;
}

```

Bagian 4: Fungsi IsolateColor (Seleksi & Border)

Ini adalah fungsi paling kompleks. Fungsinya mirip "Magic Wand Tool". Ia mencari pixel yang warnanya mirip dengan target, lalu menggambar garis putus-putus di pinggirannya.

C#

```

public Bitmap IsolateColor(Bitmap source, Color targetColor, int threshold)
{
    // ... (Setup awal sama: LockBits, Marshal.Copy, siapkan Buffer) ...

    // Buffer boolean untuk menandai pixel mana yang terpilih ("Masking")
    bool[] maskBuffer = new bool[w * h];

    // Pre-calculate target warna dan threshold kuadrat (agar tidak perlu akar kuadrat nanti, lebih cepat)
    int tR = targetColor.R; int tG = targetColor.G; int tB = targetColor.B;
    int thresholdSq = threshold * threshold;
}

```

```
int stride = srcData.Stride;
```

```
// --- TAHAP 1: MASKING ---
```

```
// Menentukan pixel mana yang masuk kriteria warna
```

```
for (int y = 0; y < h; y++)
```

```
{
```

```
    for (int x = 0; x < w; x++)
```

```
    {
```

```
        int idx = (y * stride) + (x * 4); // Rumus pointer aritmatika 2D ke 1D
```

```
        // Baca warna pixel
```

```
        byte b = srcBuffer[idx]; byte g = srcBuffer[idx + 1]; byte r = srcBuffer[idx + 2];
```

```
        // Hitung jarak warna (Euclidean Distance)
```

```
        int distR = r - tR; int distG = g - tG; int distB = b - tB;
```

```
        int distanceSq = (distR * distR) + (distG * distG) + (distB * distB);
```

```
        // Jika jarak < threshold, tandai TRUE (terpilih)
```

```
        maskBuffer[y * w + x] = (distanceSq < thresholdSq);
```

```
    }
```

```
}
```

```
// --- TAHAP 2: VISUALISASI HASIL ---
```

```
Parallel.For(0, h, y =>
```

```
{
```

```
    for (int x = 0; x < w; x++)
```

```
    {
```

```
        int idx = (y * stride) + (x * 4);
```

```
        int maskIdx = y * w + x;
```

```
        bool isSelected = maskBuffer[maskIdx]; // Cek status pixel ini
```

```
// --- LOGIKA DETEKSI TEPI (EDGE DETECTION) ---
```

```
bool isBorder = false;
```

```
if (isSelected) // Hanya cek border jika pixel ini terpilih
```

```
{
```

```
    // Cek pixel tetangga (Atas, Bawah, Kiri, Kanan)
```

```
    // Jika pixel ini TRUE, tapi tetangganya FALSE, berarti ini adalah TEPI LUKA/BATAS.
```

```
    bool top = (y > 0) ? maskBuffer[maskIdx - w] : false;
```

```
    bool bottom = (y < h - 1) ? maskBuffer[maskIdx + w] : false;
```

```
    // ... (Logic left/right sama) ...
```

```
    if (!top || !bottom || !left || !right) isBorder = true;
```

```
}
```

```

    if (isBorder)
    {
        // --- EFEK STATIC ANTS (Garis putus-putus) ---
        // Menggunakan modulus 8.
        // Jika (x+y)%8 < 4 warnanya Putih, sisanya Hitam.
        // Ini menciptakan pola garis miring.
        if ((x + y) % 8 < 4)
        {
            // Set Pixel jadi Putih
            dstBuffer[idx] = 255; dstBuffer[idx + 1] = 255; dstBuffer[idx + 2] = 255;
        }
        else
        {
            // Set Pixel jadi Hitam
            dstBuffer[idx] = 0; dstBuffer[idx + 1] = 0; dstBuffer[idx + 2] = 0;
        }
        dstBuffer[idx + 3] = 255; // Alpha Penuh
    }
    else if (isSelected)
    {
        // Jika area terpilih tapi bukan border -> Tampilkan warna asli
        dstBuffer[idx] = srcBuffer[idx];
        dstBuffer[idx + 1] = srcBuffer[idx + 1];
        dstBuffer[idx + 2] = srcBuffer[idx + 2];
        dstBuffer[idx + 3] = 255;
    }
    else
    {
        // Jika BACKGROUND (tidak terpilih) -> Ubah jadi Abu-abu (dimmed)
        byte gray = (byte)((srcBuffer[idx + 2] + srcBuffer[idx + 1] + srcBuffer[idx]) / 3);
        dstBuffer[idx] = gray;
        dstBuffer[idx + 1] = gray;
        dstBuffer[idx + 2] = gray;
        dstBuffer[idx + 3] = 255;
    }
}
});

// ... (Unlock bits & return) ...
}

```

Bagian 5: Fungsi ExtractObject (Potong)

Fungsi ini mirip IsolateColor, tapi tujuannya untuk **memotong** (crop) objek.

- Jika warna cocok: Pixel disalin (Alpha 255).
- Jika warna **tidak** cocok: Pixel dibuat transparan total (Alpha 0).

C#

```
// ...
if (distanceSq < thresholdSq)
{
    // Pixel disalin normal
    dstBuffer[idx + 3] = 255;
}
else
{
    // Pixel dimatikan (Transparan)
    dstBuffer[idx] = 0;
    // ...
    dstBuffer[idx + 3] = 0; // Kunci transparansi disini
}
// ...
```

Bagian 6: Fungsi EraseObject (Hapus)

Fungsi ini mengambil gambar asli dan "Mask Object" (hasil potongan). Dimana mask-nya ada isinya (Alpha > 0), gambar asli akan ditimpa warna Putih.

C#

```
public void EraseObject(Bitmap original, Bitmap maskObject)
{
    // ... (LockBits kedua gambar: original (ReadWrite) dan maskObject (ReadOnly)) ...
```

```

Parallel.For(0, bytes / 4, i =>
{
    int idx = i * 4;

    // Cek Alpha dari gambar Mask (Objek potongan)
    byte maskAlpha = maskBuffer[idx + 3];

    // Jika Alpha > 0 (berarti di koordinat ini ada gambar objeknya)
    if (maskAlpha > 0)
    {
        // TIMPA GAMBAR ASLI DENGAN WARNA PUTIH
        origBuffer[idx] = 255; // B
        origBuffer[idx + 1] = 255; // G
        origBuffer[idx + 2] = 255; // R
        origBuffer[idx + 3] = 255; // A
    }
});

// ... (Unlock bits) ...
}

```

Kesimpulan Konsep Utama

1. **Manipulasi Byte Array:** Kode ini menghindari GetPixel yang lambat dengan menyalin seluruh data gambar ke byte[], memanipulasinya, lalu mengembalikannya.
2. **Euclidean Distance:** Digunakan untuk menentukan seberapa mirip sebuah warna dengan warna target.
3. **Alpha Channel:** Memainkan byte ke-4 (Alpha) sangat penting untuk membuat transparansi saat ekstraksi objek.
4. **Parallel Processing:** Menggunakan Parallel.For membuat operasi berat ini berjalan lancar secara *real-time*.

Apakah Anda ingin saya menjelaskan lebih lanjut tentang rumus matematika di ColorMapType.Jet atau tentang logika deteksi tepinya?